

FinWise AI Project Report

SMART PERSONAL FINANCE MANAGEMENT SYSTEM

ABSTRACT

FinWise AI is a comprehensive, offline-first personal finance management system designed to assist users in tracking their educational and professional financial transactions, managing budgets, and planning savings goals. Traditional finance apps depend on cloud databases and external servers, raising concerns about data privacy and internet reliability. FinWise AI resolves these issues by utilizing a local-first deployment model where user records, statements, and intelligent advice are processed entirely on the client's machine.

Central to the system is a local intelligent categorizer based on a **Multinomial Naive Bayes** category prediction model that matches transaction descriptions to categories without sending transaction logs to third-party services. Additionally, an offline rule-based chat assistant answers finance-related queries, retrieves real-time database summaries, and offers actionable financial insights.

Using modern web technologies—a **React** frontend, a **Flask (Python)** backend, and a **MySQL** database—FinWise AI separates responsibilities through a classic three-tier architecture. The system supports multi-account ledgers (Bank, Cash, UPI, Loans, Credit Cards, Investments), bank statement imports (CSV, Excel, text-based PDF) with automatic duplicate detection, and visual month-wise analytics. By consolidating accounting principles with local artificial intelligence, FinWise AI significantly reduces manual tracking workloads, secures private financial data, and enables users to establish sound budgets and savings habits.

TABLE OF CONTENTS

S.NO	TITLE	PAGE NO.		---		---		---		ABSTRACT	**	(i)	**		LIST OF FIGURES	**	(ii)	**		LIST OF TABLES	**	(iii)	**		CHAPTERS	**	TITLE	**	PAGE NO.	**	1	**	INTRODUCTION	**	01	**																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																		
										1.1 PROJECT INTRODUCTION		01		2	**	WORKING ENVIRONMENT	**	02	**		2.1 HARDWARE REQUIREMENT		02				2.2 SOFTWARE REQUIREMENT		02				2.3 SYSTEM SOFTWARE		03		3	**	SYSTEM ANALYSIS	**	05	**		3.1 FEASIBILITY STUDY		05				3.2 EXISTING SYSTEM		05				3.3 DRAWBACKS OF EXISTING SYSTEM		06				3.4 PROPOSED SYSTEM		06				3.5 BENEFITS OF PROPOSED SYSTEM		07				3.6 SCOPE OF THE PROJECT		07		4	**	SYSTEM DESIGN	**	08	**				4.1 SYSTEM ARCHITECTURE		08				4.2 SYSTEM DIAGRAMS (USE CASE, CLASS, SEQUENCE, ERD, DFD)		08				4.3 DATABASE DESIGN		12		5	**	PROJECT DESCRIPTION	**	14	**				5.1 OBJECTIVE		14				5.2 MODULE DESCRIPTION		14				5.3 IMPLEMENTATION & ALGORITHMS		16				5.4 MAINTENANCE STRATEGY		18		6	**	SYSTEM TESTING	**	19	**				6.1 TESTING DEFINITION		19				6.2 TESTING OBJECTIVE		19				6.3 TYPES OF TESTING		20				6.4 TEST CASES		21		7	**	CONCLUSION	**	23	**				7.1 SUMMARY		23				7.2 FUTURE ENHANCEMENT		23		8	**	APPENDIX	**	25	**				8.1 SCREENSHOTS & USER INTERFACE		25				8.2 KEY CODING MODULES		26				8.3 DATA DICTIONARY		32		9	**	BIBLIOGRAPHY AND REFERENCES	**	35	**																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																

LIST OF FIGURES

FIGURE NO	TITLE	PAGE NO	---	---	---	4.1	SYSTEM ARCHITECTURE DIAGRAM	08	4.2	USE CASE DIAGRAM	09	4.3	CLASS DIAGRAM	10	4.4	SEQUENCE DIAGRAM	11	4.5	ENTITY RELATIONSHIP DIAGRAM (ERD)	12	4.6	DATA FLOW DIAGRAM (DFD)	12
-----------	-------	---------	-----	-----	-----	-----	-----------------------------	----	-----	------------------	----	-----	---------------	----	-----	------------------	----	-----	-----------------------------------	----	-----	-------------------------	----

LIST OF TABLES

TABLE NO TITLE PAGE NO	:--- :--- :---	2.1 HARDWARE REQUIREMENTS 02	2.2 SOFTWARE REQUIREMENTS 02	6.4 SYSTEM TEST CASES 21	8.3 DATABASE DATA DICTIONARY 32
----------------------------	--------------------	----------------------------------	----------------------------------	------------------------------	-------------------------------------

1. INTRODUCTION

1.1 Project Introduction

Managing personal finances is a foundational skill for students, working professionals, and families alike. In today's economy, individuals frequently move money across multiple channels, including traditional savings accounts, cash on hand, e-wallets, UPI accounts, and credit cards. When loans and investment balances are factored in, tracking financial health becomes challenging.

Most people manage money using manual physical ledgers, spreadsheets, or online banking applications. However, manual methods are error-prone and time-consuming, spreadsheets lack dynamic visualization and intelligent categorization, and online applications raise significant concerns regarding user data privacy and require constant internet connectivity.

FinWise AI is a desktop personal finance management system that enables offline tracking of multiple account types under one interface. The system acts as a digital ledger that records transactions (Income, Expense, Transfer) and dynamically maintains individual account balances.

Furthermore, the application integrates offline machine learning through a local category prediction system using **Multinomial Naive Bayes** to classify transactions based on description text (e.g., "Swiggy" automatically categorized as "Food"). An offline rule-based chat assistant responds to user queries about their accounts, budgets, and savings goals using the local SQLite/MySQL data directly on the client machine. This provides a secure, private, internet-independent dashboard for comprehensive personal finance management.

2. WORKING ENVIRONMENT

2.1 Hardware Requirement

For the FinWise AI platform to run successfully, both the web browser client and the local database and application servers require standard computing hardware. Because all AI components run locally without connecting to external cloud servers, the local machine handles all prediction calculations.

Table 2.1 Hardware Requirements

Component What We Used Why It Works	:--- :--- :---	Processor (CPU) Intel Core i5 (11th Gen, 64-bit, 2.4 GHz) or comparable AMD Ryzen 5 Enables efficient execution of the local Flask server, background classification threads, and database transactions.	RAM (Memory) 8 GB DDR4 (16 GB recommended) Supports running the React web browser client, the Flask API backend, and the MySQL database engine concurrently.	Storage 512 GB Solid-State Drive (SSD) Minimizes read/write latency for transaction logs, local uploads (receipts), and rapid SQL queries.	Network (LAN/WLAN) Not required for execution (Localhost) Essential only during initial setup (installing npm and pip packages). The system runs entirely offline post-installation.
---	--------------------	---	---	---	---

2.2 Software Requirement

The following software components are required to compile, host, and test FinWise AI:

Table 2.2 Software Requirements

| Name | Details | | :--- | :--- | | ****Operating System**** | Windows 10 / Windows 11 (64-bit) | | ****Database Server**** | MySQL Community Server (v8.0+) | | ****Frontend Framework**** | React.js (v18+) with Vite, Bootstrap CSS | | ****Backend Framework**** | Python (v3.10+) with Flask | | ****Runtime Libraries**** | MySQL Connector Python, PyJWT (JSON Web Tokens) | | ****File Processors**** | openpyxl, xlrd (Excel processing), pdfplumber (PDF statements) | | ****Report Generator**** | reportlab (PDF generation engine) | | ****Browser Environment**** | Google Chrome, Microsoft Edge, or Mozilla Firefox |

2.3 System Software

Operating System (OS)

Windows 11 is used as the operating system for local development, testing, and deployment. The OS manages system calls, schedules process threads, coordinates file access for local receipt storage, and manages socket connections between the frontend and backend servers via loopback IP (`127.0.0.1`).

HTML5 & CSS3

HTML5 provides the structured framework of the user interface. Semantic elements (main, aside, section, and nav) organize the screens. CSS3, extended via Bootstrap, implements responsive styling, custom grids, animations, and transitions for light, dark, and system color themes.

React.js

React is an open-source JavaScript library utilized to build single-page interfaces (SPAs). Its component-based architecture enables component reuse and local state tracking using hooks (e.g., `useState`, `useEffect`). Only parts of the page reload when transaction entries, budgets, or chat windows update, resulting in a fast user experience.

Bootstrap & Bootstrap Icons

Bootstrap provides standardized CSS grid structures, form designs, tables, modals, and buttons. Interactive visual components match the system's dark/light modes. Bootstrap Icons add context symbols to dashboard tiles and side nav menus.

Python & Flask

Python serves as the main programming language for backend operations. Flask is a micro-framework that implements the REST API routes. Flask handles user authentication, data queries, transactions, exports, imports, and execution of the local Naive Bayes classifier.

MySQL (Database)

MySQL stores relational data including user profiles, transactions, accounts, budgets, and savings goals. It enforces relational integrity through foreign keys and supports transactions via transactional SQL queries to prevent database corruption during balance updates.

PyJWT

JSON Web Tokens (JWT) manage secure user login sessions. Upon authentication, Flask generates a signed token containing the user's identity. The token is stored in the client's browser as a secure, httpOnly cookie to prevent Cross-Site Scripting (XSS) data leaks.

openpyxl, xlrd, & pdfplumber

These utilities parse imported bank statements. `openpyxl` and `xlrd` parse Excel worksheets, while `pdfplumber` extracts text strings from digital PDFs to read raw dates, amounts, and transaction descriptions.

reportlab

`reportlab` is a Python library that builds custom PDF files. It formats database records into tables, calculates totals, and renders an exportable document structure for the user.

3. SYSTEM ANALYSIS

3.1 Feasibility Study

Technical Feasibility

The platform uses well-supported web tools: React, Flask, and MySQL. Since all processing runs locally on the host machine, no remote GPU clusters or premium API keys are required. The Naive Bayes algorithm is implemented natively in Python without heavy external libraries, confirming the project's technical feasibility.

Operational Feasibility

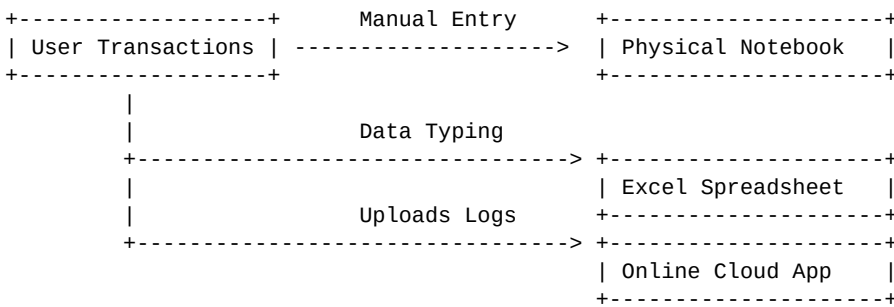
FinWise AI features an intuitive layout with a sidebar menu, transaction forms, and budget cards. The AI assistant functions as a chat box, making it easy for users to find help without learning complex syntax. This simple user experience confirms operational feasibility.

Economic Feasibility

The platform is built using free, open-source development packages. Because it runs on the user's local machine, there are no ongoing cloud hosting, cloud database, or AI API fees. This makes the project highly cost-effective and economically feasible.

3.2 Existing System

Currently, most individuals manage their money through manual bookkeeping, offline spreadsheets, or cloud-connected budgeting apps.



In the existing setup:

- Notebook entries require manually calculating running balances after every transaction.
- Spreadsheet users must manually write formulas to link transactions, accounts, and goal trackers.
- Cloud apps automatically read bank details by linking accounts to third-party servers. However, they raise privacy concerns, are subject to cloud server downtime, and require a constant internet connection.

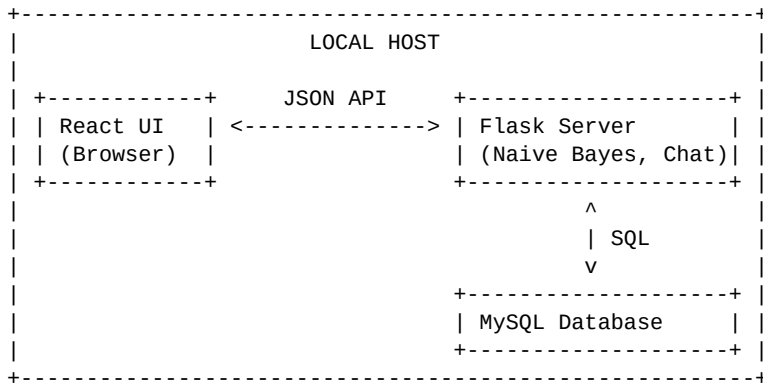
3.3 Drawbacks of Existing System

1. **Lack of Centralization**: Cash, UPI wallets, credit cards, and loan balances are tracked across separate apps, preventing a unified view of total funds.
2. **Data Security and Privacy Risks**: Online budget apps store transaction histories, merchant names, and income details on cloud servers, exposing them to potential data leaks.
3. **No Automatic Categorization (Offline)**: Spreadsheet users must manually assign categories to every transaction, increasing the manual tracking workload.

4. **Internet Dependency**: Cloud services are unusable without cellular network or Wi-Fi access, limiting accessibility.
5. **Complicated Account Balances**: Existing systems often struggle to track savings goals without counting them as standard expenses, which misrepresents available balances.
6. **No Real-Time Guidance**: Static ledger books and spreadsheets do not alert users when budgets are close to being exceeded or provide on-demand answers to financial questions.

3.4 Proposed System

The proposed system, **FinWise AI**, is an offline-first financial dashboard that runs on the user's local machine.



The system addresses the limitations of the existing methods:

- **Centralized Ledger**: Consolidates bank accounts, cash, UPI wallets, investments, loans, and credit cards into a single database.
- **Privacy and Security**: All transaction histories and account details remain stored in a local MySQL instance on the user's hard drive.
- **Multinomial Naive Bayes Categorizer**: Learns from transaction descriptions to predict categories, reducing manual entry requirements.
- **Offline Chat Assistant**: Answers financial questions and analyzes current data directly from the local MySQL database.
- **Statement Import & Duplicate Checking**: Imports CSV, Excel, and PDF bank statements, detects matching records, and prevents duplicate entries.

3.5 Benefits of Proposed System

- **Offline Execution**: Operates without active internet access after initial setup.
- **Local Data Control**: Secures personal financial information on the host computer.
- **Explainable AI**: The categorization and chat systems use transparent algorithms that run locally.
- **Double Ledger Balance Tracking**: Automatically creates debit and credit entries to keep accounts aligned.
- **Smart Tools and Reminders**: Automates recurring transactions (e.g., salary, rent) to prevent manual entry errors.

3.6 Scope of the Project

The scope of FinWise AI is focused on providing offline personal finance tracking for a single user at a time:

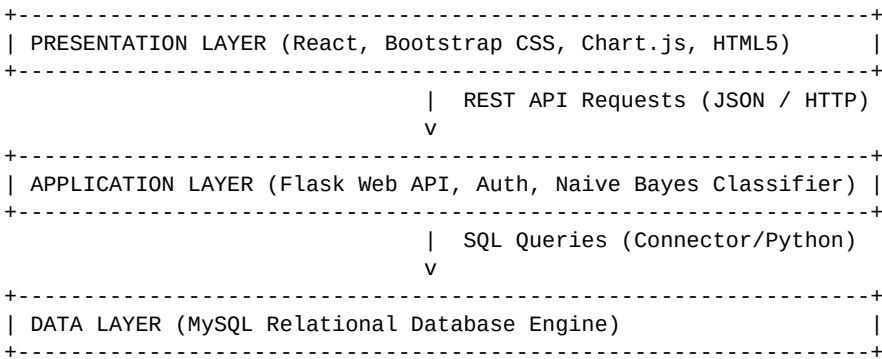
- Support for six account types: Bank, Cash, UPI Wallet, Credit Card, Loan, and Investment.
- Automated file processing for statement uploads (CSV, Excel, and text-based PDFs).

- Automated duplicate check comparisons using transaction metadata.
- Local help chatbot that reads current database statistics to answer queries.
- Responsive design with light, dark, and system-level themes.
- Monthly budget warnings, savings goal progress tracking, and visual analytics reports.

4. SYSTEM DESIGN

4.1 System Architecture

FinWise AI utilizes a standard **Three-Tier Architecture** to separate user interaction, business logic, and data storage:

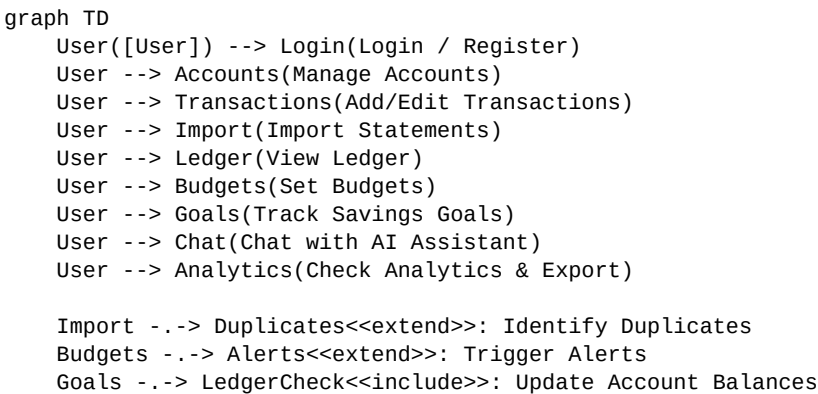


- **Presentation Layer**: Built with React, Vite, Bootstrap, and Chart.js. Renders pages, forms, and charts, handles styling themes, and manages local component states.
- **Application Layer**: Flask REST API web server. Coordinates authentication, routes requests, processes imported statements, and executes the Naive Bayes categorization and chat matching.
- **Data Layer**: MySQL database. Stores records and handles referential constraints (e.g., deleting a user removes associated accounts).

4.2 System Diagrams

Use Case Diagram

The Use Case Diagram displays system interactions for the primary user.



Class Diagram

The Class Diagram illustrates the data structures and relationships within the system.

```
classDiagram
```

```

class User {
    +int id
    +string name
    +string email
    +string password
    +register()
    +login()
    +editProfile()
}
class Account {
    +int id
    +string name
    +string type
    +decimal current_balance
    +decimal opening_balance
    +reconcile(actual)
}
class Transaction {
    +int id
    +string type
    +string category
    +decimal amount
    +date transaction_date
    +string description
    +string review_status
    +create()
    +split()
}
class Budget {
    +int id
    +string category
    +string month_year
    +decimal budget_amount
    +checkLimit()
}
class SavingsGoal {
    +int id
    +string goal_name
    +decimal target_amount
    +decimal saved_amount
    +addContribution()
}
class StatementImport {
    +int id
    +string file_name
    +string file_hash
    +int imported_rows
    +detectDuplicates()
}

User "1" --> "*" Account : owns
User "1" --> "*" Transaction : records
User "1" --> "*" Budget : targets
User "1" --> "*" SavingsGoal : tracks
User "1" --> "*" StatementImport : uploads
Account "1" --> "*" Transaction : registers
StatementImport "1" --> "*" Transaction : generates

```

Sequence Diagram

The Sequence Diagram outlines the timeline flow of transactions, from user entry to the Naive Bayes classification.

```

sequenceDiagram
    autonumber
    actor User
    participant Frontend as React Frontend
    participant Backend as Flask Backend
    participant DB as MySQL DB

```

```

User-->Frontend: Input transaction description
Frontend-->Backend: POST /api/ai/categorize {description}
Backend-->DB: Query transaction history
DB-->Backend: Return category training rows
Backend-->Backend: Compute Naive Bayes probabilities
Backend-->Frontend: Return predicted category & confidence
Frontend-->User: Populate category field in form
User-->Frontend: Submit transaction
Frontend-->Backend: POST /api/transactions
Backend-->DB: INSERT transaction & UPDATE account balance
DB-->Backend: Acknowledge commit
Backend-->Frontend: Success response
Frontend-->User: Refresh Dashboard & Ledger

```

Activity Diagram

The Activity Diagram displays the workflow of importing a statement file.

```

stateDiagram-v2
    [*] --> ChooseFile
    ChooseFile --> SelectAccount
    SelectAccount --> ParseFile : CSV, Excel, or PDF
    ParseFile --> CheckDuplicates : Match date, amount, description
    state CheckDuplicates {
        [*] --> CompareRows
        CompareRows --> MatchFound : Similarity > Threshold
        CompareRows --> UniqueRow : No matching entries
    }
    UniqueRow --> PreviewGrid : Display as Cleared
    MatchFound --> PreviewGrid : Flag as Duplicate
    PreviewGrid --> UserEdits : Modify categories/details
    UserEdits --> SaveAll : User confirms
    SaveAll --> UpdateBalances : Recalculate account balances
    UpdateBalances --> [*]

```

Entity Relationship Diagram (ERD)

The ERD shows the relational database schema of the MySQL tables.

```

erDiagram
    USERS ||--o{ ACCOUNTS : "1 to N (user_id)"
    USERS ||--o{ TRANSACTIONS : "1 to N (user_id)"
    USERS ||--o{ BUDGETS : "1 to N (user_id)"
    USERS ||--o{ SAVINGS_GOALS : "1 to N (user_id)"
    USERS ||--o{ STATEMENT_IMPORTS : "1 to N (user_id)"
    ACCOUNTS ||--o{ TRANSACTIONS : "1 to N (account_id)"
    ACCOUNTS ||--o{ GOAL_CONTRIBUTIONS : "1 to N (account_id)"
    SAVINGS_GOALS ||--o{ GOAL_CONTRIBUTIONS : "1 to N (goal_id)"
    STATEMENT_IMPORTS ||--o{ TRANSACTIONS : "1 to N (statement_import_id)"

```

Data Flow Diagram (DFD)

The DFD displays how data moves from user inputs through process transforms to the database.

```

graph TD
    User((User)) -- "1. Form Actions" --> UI[React Frontend UI]
    UI -- "2. HTTP API Request" --> Flask[Flask Web Server]

    subgraph Processes [Flask Server Core]
        Auth[3.1 JWT Auth Validator]
        Ledger[3.2 Ledger Balance Engine]
        AI[3.3 Naive Bayes Prediction Engine]
        Chat[3.4 Chat Query Matching]
    end

    Flask --> Auth
    Flask --> Ledger
    Flask --> AI
    Flask --> Chat

```

```
Auth -- "Validate Session" --> DB[(MySQL Relational Database)]
Ledger -- "Write Transaction & Update Balances" --> DB
AI -- "Train & Predict Category" --> DB
Chat -- "Read Account Snapshot" --> DB

DB -- "Record Sets" --> Flask
Flask -- "JSON response" --> UI
UI -- "3. Dashboard / Visual Reports" --> User
```

4.3 Database Design

FinWise AI implements a relational database structure designed to support transactional updates:

1. users

Tracks user account credentials and registration status. Passwords are encrypted using script hashes to prevent unauthorized access in the database.

- Columns: `id` (INT, PK), `name` (VARCHAR), `email` (VARCHAR, Unique), `password` (VARCHAR), `created\_at` (TIMESTAMP).

2. accounts

Maintains balances and account configurations for multiple asset classes.

- Columns: `id` (INT, PK), `user\_id` (INT, FK), `account\_name` (VARCHAR), `account\_type` (ENUM), `opening\_balance` (DECIMAL), `current\_balance` (DECIMAL), `credit\_limit` (DECIMAL), `created\_at` (TIMESTAMP).

3. transactions

Stores income, expenses, and transfers.

- Columns: `id` (INT, PK), `user\_id` (INT, FK), `account\_id` (INT, FK), `type` (ENUM), `category` (VARCHAR), `amount` (DECIMAL), `description` (VARCHAR), `transaction\_date` (DATE), `transfer\_to\_account` (INT, Null), `review\_status` (ENUM), `created\_at` (TIMESTAMP).

4. budgets

Stores category-wise spending limits for a given month.

- Columns: `id` (INT, PK), `user\_id` (INT, FK), `category` (VARCHAR), `month\_year` (VARCHAR), `budget\_amount` (DECIMAL), `created\_at` (TIMESTAMP).

5. savings_goals

Maintains long-term savings goals and target progress.

- Columns: `id` (INT, PK), `user\_id` (INT, FK), `goal\_name` (VARCHAR), `target\_amount` (DECIMAL), `saved\_amount` (DECIMAL), `deadline` (DATE), `created\_at` (TIMESTAMP).

6. goal_contributions

Records movements of money into and out of savings goals, tracking the source accounts.

- Columns: `id` (INT, PK), `user\_id` (INT, FK), `goal\_id` (INT, FK), `account\_id` (INT, FK), `action` (ENUM), `amount` (DECIMAL), `created\_at` (TIMESTAMP).

7. statement_imports

Tracks details of imported statement files to prevent duplicate imports.

- Columns: `id` (INT, PK), `user\_id` (INT, FK), `account\_id` (INT, FK), `file\_name` (VARCHAR), `file\_hash` (CHAR(64)), `imported\_rows` (INT), `created\_at` (TIMESTAMP).

5. PROJECT DESCRIPTION

5.1 Objective

The main objective of FinWise AI is to build a secure, private, and offline-first personal finance system. The project targets three key goals:

1. **Consolidate Tracking**: Provide a single view of financial records, including cash, bank accounts, loans, and credit cards.
2. **Automate Input Processing**: Minimize manual data entry through bank statement uploads and text prediction algorithms.
3. **Data Security and Privacy**: Ensure all financial logs, statements, and AI insights are kept secure on the user's local hardware.

5.2 Module Description

5.2.1 Authentication and Profile

Manages registration, secure login, profile edits, and color theme preferences.

- **Key Functions**: Password hashing via bcrypt, httpOnly session cookies, and local profile changes.
- **Impact**: Restricts access to authorized users and isolates data when multiple users use the same application source.

5.2.2 Accounts Ledger

Tracks financial accounts, updating balances automatically when transactions occur.

- **Key Functions**: Supports multiple account types (Bank, Cash, UPI Wallet, Credit Card, Loan, Investment) with automatic balance adjustments.
- **Impact**: Provides users with an accurate, real-time calculation of their available balance.

5.2.3 Manual Transactions

Enables manual entry of transaction records.

- **Key Functions**: Records manual transactions as income, expenses, or transfers.
- **Impact**: Modifies account balances and provides the data used for category prediction and analytics.

5.2.4 Smart Statement Import

Allows users to upload statement files (CSV, Excel, or PDF).

- **Key Functions**: Parses statement files, displays a preview grid for manual corrections, and checks files against database hashes.
- **Impact**: Saves time by allowing batches of transactions to be reviewed and imported at once.

5.2.5 Ledger Engine

Generates detailed transaction lists with running balances.

- **Key Functions**: Implements the running balance formula and records transfers as matching debit and credit entries.

- **Impact**: Provides an auditable view of transaction histories to verify how specific transactions impacted account balances.

5.2.6 Budget Planner

Tracks spending limits against actual monthly expenses.

- **Key Functions**: Defines category limits and checks transaction data to monitor budget utilization.
- **Impact**: Alerts users when they are close to or over budget in specific spending categories.

5.2.7 Savings Goal Tracker

Tracks financial targets by separating saved goal contributions from general spending.

- **Key Functions**: Manages deposit/withdrawal contributions and updates progress percentages.
- **Impact**: Helps users save money for specific goals without misrepresenting available cash as general expenses.

5.2.8 Analytics Dashboard

Provides month-by-month financial summaries using interactive charts.

- **Key Functions**: Groups transaction categories, computes net cash flow, and renders visual charts.
- **Impact**: Helps users identify trends and analyze their largest spending categories.

5.2.9 Smart Tools & Local Automation

Manages automation tasks and recurring transaction rules.

- **Key Functions**: Schedules automatic entries (e.g., salary, subscription bills) and monitors accounts for duplicate entries.
- **Impact**: automates the entry of recurring transactions, reducing manual tracking errors.

5.2.10 Offline AI Chat Assistant

An offline conversational assistant that answers financial questions.

- **Key Functions**: Maps keywords to help topics and retrieves current account statistics.
- **Impact**: Explains financial terms and provides database summaries without requiring external API requests.

5.3 Implementation & Algorithms

1. Multinomial Naive Bayes Category Classifier

When a user imports a statement or enters a description, the classifier predicts the transaction category.

Mathematical Formulation Let $C = \{c_1, c_2, \dots, c_k\}$ be the set of possible transaction categories (e.g., Food, Travel, Bills, Salary). Let D be the input description represented as a bag-of-words token vector $W = \{w_1, w_2, \dots, w_n\}$.

Under Bayes' Theorem, the probability of category c_j given the words W is: $P(c_j \mid W) = \frac{P(c_j) \prod_{i=1}^n P(w_i \mid c_j)}{P(W)}$

Since $P(W)$ is constant across all categories, we maximize the numerator: $\text{Predicted Category} = \arg\max_{c_j \in C} P(c_j) \prod_{i=1}^n P(w_i \mid c_j)$

To prevent underflow issues with floating-point math, we calculate the sum of logs: $\log P(c_j \mid W) \propto \log P(c_j) + \sum_{i=1}^n \log P(w_i \mid c_j)$

Smoothing and Training To handle words not seen during training, we use Laplace smoothing: $P(w_i | c_j) = \frac{N_{c_j, w_i} + 1}{N_{c_j} + V}$

Where:

- N_{c_j, w_i} is the frequency of word w_i in category c_j .
- N_{c_j} is the total count of all words in category c_j .
- V is the size of the vocabulary across all categories.

The training set combines a pre-defined dictionary of common financial terms with the user's historical transactions. If the calculated confidence score falls below a set threshold (e.g., 45%), the system defaults to "Other" to prevent incorrect predictions.

2. Rule-Based Chat Assistant

The assistant maps user queries to specific help responses or database statistics using a keyword-matching score.

```
Question String: "What is my current bank balance?"
    |
    v
Tokenizer (Split to lowercase words: ['what', 'is', 'my', 'current', 'bank', 'balance'])
    |
    v
Keyword Scorer (Cross-reference with help topics: 'balance' => 2 points, 'bank' => 2 points)
    |
    v
Intent Match (Select topic with highest score)
    |
    v
Database Query (Retrieve current balances from accounts table)
    |
    v
Format Response: "Your total bank balance is Rs 45,500 across 2 accounts."
```

3. Ledger Running Balance Algorithm

The system processes transactions chronologically to calculate a running balance: $\text{Balance}_t = \text{Balance}_{t-1} + \text{Credit}_t - \text{Debit}_t$

For transfers, the system updates both the source and target accounts:

- Decrements the source account balance (Debit).
- Increments the target account balance (Credit).

4. Duplicate Transaction Detection

Matches pending statement records against existing entries. The similarity check evaluates: $\text{Match} = (\text{Account}_A == \text{Account}_B) \wedge (\text{Date}_A == \text{Date}_B) \wedge (\text{Amount}_A == \text{Amount}_B) \wedge (\text{Type}_A == \text{Type}_B)$

If a match is found, the row is flagged to prevent duplicate imports.

5.4 Maintenance Strategy

1. Corrective Maintenance

Focuses on identifying and resolving runtime bugs. Examples include fixing formatting errors in statement parsers and correcting calculations in the ledger balance loop.

2. Adaptive Maintenance

Updates the application to maintain compatibility with updated browsers and library versions (e.g., Python, Flask, or React packages) without impacting core offline functionality.

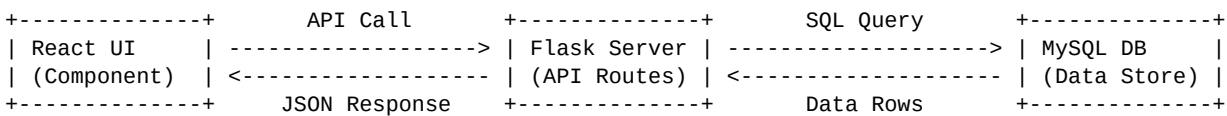
3. Perfective Maintenance

Implements performance optimizations, such as indexing database columns to improve query speeds and refining the Naive Bayes algorithm to improve categorization accuracy.

6. SYSTEM TESTING

6.1 Testing Definition

System testing evaluates the integrated FinWise AI application to verify it meets design requirements. The testing process checks frontend components, backend API logic, and the local database connection as a unified system.



Testing confirms the system operates correctly under standard conditions, handles input errors gracefully, protects user sessions, and manages account balances consistently.

6.2 Testing Objective

The primary testing objectives are:

1. ****Verification of Calculations****: Confirm the database correctly calculates balances, budgets, and goals after transaction changes.
2. ****Evaluation of AI Accuracy****: Verify the Naive Bayes classifier predicts correct categories for test descriptions.
3. ****Session Verification****: Confirm httpOnly cookies block unauthorized API requests.
4. ****Validation of File Parsing****: Ensure the CSV, Excel, and PDF statement processors parse transaction rows without errors.
5. ****Confirmation of Duplicate Detection****: Verify the system flags duplicate rows during statement imports.

6.3 Types of Testing

Unit Testing

Evaluates individual functions in isolation. Tests check the Naive Bayes scoring calculations, password hashing, and text parsing functions.

Integration Testing

Checks communication between different modules. Tests verify that registering a transaction updates the transaction table and modifies the corresponding account balance.

System Testing

Runs end-to-end user workflows. Tests verify registration, manual transaction logging, statement imports with duplicates, and database exports.

User Acceptance Testing (UAT)

Verifies the application UI meets student usability requirements. Feedback is collected on layout readability, dark mode visibility, and the helpfulness of the chat assistant.

Compatibility Testing

Confirms the responsive Bootstrap interface compiles and renders correctly across common web browsers, including Google Chrome, Microsoft Edge, and Mozilla Firefox.

6.4 Test Cases

The following test suite verifies the core features of the platform:

Table 6.4 System Test Cases

Test Case ID	Test Scenario	Test Steps / Input	Expected Output	Actual Result	Status
TC001	User Registration	Submit a registration form with unique credentials.	User account created and saved in the database.	Account saved successfully.	Pass
TC002	User Login	Enter valid credentials.	JWT token generated and stored in a secure cookie.	Session token stored.	Pass
TC003	Invalid Login	Enter an incorrect password.	Display an unauthorized access error.	Login rejected, error shown.	Pass
TC004	Add Transaction	Create an expense of Rs 500 from a bank account.	Decrements the account balance by Rs 500.	Balance decreased by Rs 500.	Pass
TC005	Double Ledger Transfer	Transfer Rs 1,000 from Bank to UPI Wallet.	Decrements Bank by Rs 1,000 and increments UPI by Rs 1,000.	Balances updated.	Pass
TC006	Naive Bayes Prediction	Input transaction description: "Swiggy lunch".	Predicts the "Food" category with high confidence.	Category populated as "Food".	Pass
TC007	Statement Import	Upload a bank statement file.	Parses transaction rows and displays a preview grid.	Grid displays parsed rows.	Pass
TC008	Duplicate Detection	Upload a statement containing an existing transaction.	Flags the duplicate row to prevent duplicate import.	Row flagged.	Pass
TC009	Budget Tracking	Enter an expense that exceeds 80% of a category budget.	Triggers a budget alert warning on the dashboard.	Alert warning displayed.	Pass
TC010	Savings Goal Tracker	Contribute Rs 2,000 to a savings goal.	Deducts balance from source account and increases goal savings.	Goal savings updated.	Pass
TC011	Chat Query Processing	Submit query: "what is Naive Bayes?".	Returns the help guide explaining the local ML classifier.	Guide text displayed.	Pass
TC012	PDF Report Export	Click the export button.	Generates and downloads a formatted financial report PDF.	PDF downloaded.	Pass

7. CONCLUSION

7.1 Summary

FinWise AI is a complete, offline-first personal finance management system. The application coordinates financial tracking across multiple accounts, including bank accounts, cash, UPI wallets, credit cards, investments, and loans.

By utilizing local data management and custom classification algorithms, the system provides personal finance tools while keeping user data secure.

Key achievements of the platform include:

- A double-entry accounting ledger that manages running balances.
- Local category prediction using a Multinomial Naive Bayes classifier.
- An offline help assistant that retrieves database statistics on demand.
- A statement import workflow with built-in duplicate detection.

The application operates without internet dependencies, protecting financial privacy and ensuring accessibility.

7.2 Future Enhancement

Future updates could extend the platform's capabilities:

1. **Machine Learning Forecasting**: Integrate regression models to forecast future utility expenses based on seasonal usage trends.
2. **Optical Character Recognition (OCR)**: Integrate an offline OCR library to extract transaction details directly from images of receipts.
3. **Automatic Statement Synchronization**: Build secure integrations with open banking APIs to allow optional statement updates when internet access is available.
4. **Advanced Calculators**: Add interactive planning tools for loan EMI schedules and credit card interest calculations.
5. **Mobile Application**: Build a React Native client that shares the local database schema, enabling mobile tracking.

8. APPENDIX

8.1 Screenshots & User Interface

The application interface is divided into functional sections managed from the sidebar:

8.1.1 Dashboard View

The main landing page displays financial summary cards:

- **Total Money**: Combined account balances and goal savings.
- **Balance Money**: Available cash and bank funds.
- **Monthly Net Cash Flow**: Income vs expenses.
- **Visual Panels**: Displays recent transactions, budget utilization bars, and a quick-entry form.

8.1.2 Accounts Manager

A grid showing configured financial accounts:

- Displays account details including type, last 4 digits of the account number, and current balances.
- Includes a button to open the account reconciliation form.

8.1.3 Transactions Ledger

A detailed table of financial transactions:

- Displays date, type, account, category, amount, and descriptions.
- Features controls to filter transactions, split entries, and import statement files.

8.1.4 Budget Planner

A budget configuration dashboard:

- Displays progress bars showing actual spending against monthly budget limits.
- Highlights over-budget categories in red.

8.1.5 Savings Goal Tracker

A visual dashboard tracking savings progress:

- Displays progress bars showing saved amounts against goal targets.
- Includes forms to log goal deposits and withdrawals.

8.1.6 Analytics Center

An interactive analytics view:

- Displays spending breakdowns by category using pie and bar charts.
- Shows historical income vs expense trends over a six-month period.

8.1.7 Offline Chat Assistant

A conversational chat interface:

- Includes a scrollable chat log displaying responses and suggested follow-up chips.
- Processes user queries locally, retrieving database statistics without external server requests.

8.2 Key Coding Modules

8.2.1 Frontend Authentication Wrapper (`frontend/src/main.jsx`)

Coordinates app initialization and user routing:

```
import React, { useEffect, useState } from "react";
import { createRoot } from "react-dom/client";
import "bootstrap/dist/css/bootstrap.min.css";
import "bootstrap-icons/font/bootstrap-icons.css";
import "./styles/app.css";
import api from "../services/api";
import Login from "../pages/Login";
import Register from "../pages/Register";
import Dashboard from "../pages/Dashboard";

function App() {
  const [authenticated, setAuthenticated] = useState(null);
  const [page, setPage] = useState("login");

  useEffect(() => {
    // Check user session
    api.get("/profile")
      .then(() => setAuthenticated(true))
      .catch(() => setAuthenticated(false));
  }, []);

  if (authenticated === null) {
    return (
      <div className="auth-bg">
        <div className="auth-card shadow text-center">
          <h4>Loading FinWise...</h4>
        </div>
      </div>
    );
  }

  if (!authenticated && page === "register") {
    return <Register setPage={setPage} />;
  }

  if (!authenticated) {
    return <Login setToken={() => setAuthenticated(true)} setPage={setPage} />;
  }

  return <Dashboard setToken={setAuthenticated} />;
}

createRoot(document.getElementById("root")).render(<App />);
```

8.2.2 Relational Database Interface (`backend/db.py`)

Manages MySQL connection pooling and query execution:

```
import mysql.connector
from config import DB_CONFIG

def get_conn():
    """Establishes and returns a connection to the MySQL database."""
    return mysql.connector.connect(**DB_CONFIG)

def query(sql, params=None, fetch=False, one=False):
    """Executes a SQL query and handles connection lifecycle."""
    conn = get_conn()
    cur = conn.cursor(dictionary=True)
    cur.execute(sql, params or ())

    if fetch:
        data = cur.fetchone() if one else cur.fetchall()
    else:
        data = None

    conn.commit()
    cur.close()
    conn.close()
    return data
```

8.2.3 Session Authentication and Middleware (`backend/auth.py`)

Secures REST endpoints using JWT tokens:

```
import hashlib
import jwt
import uuid
from datetime import datetime, timezone, timedelta
from functools import wraps
from flask import request, jsonify
from werkzeug.security import generate_password_hash, check_password_hash
from config import JWT_SECRET, ACCESS_TOKEN_MINUTES, REFRESH_TOKEN_DAYS
from db import query

def hash_password(password):
    """Encrypts raw passwords using scrypt hashing."""
    return generate_password_hash(password, method="scrypt")

def verify_password(stored, password):
    """Verifies input passwords against stored scrypt hashes."""
    if not stored:
        return False
    if stored.startswith(("scrypt:", "pbkdf2:")):
        return check_password_hash(stored, password)
    return stored == hashlib.sha256(password.encode()).hexdigest()

def token_required(f):
    """Decorator to protect Flask endpoints using JWT verification."""
    @wraps(f)
    def wrapper(*args, **kwargs):
        auth = request.headers.get('Authorization', '')
        token = auth.split(' ', 1)[1] if auth.startswith('Bearer ') else request.cookies.get('access_token')

        if not token:
            return jsonify({'error': 'Token missing'}), 401

        try:
            data = jwt.decode(token, JWT_SECRET, algorithms=["HS256"])
            if data.get("type") != "access":
                return jsonify({'error': 'Wrong token type'}), 401

            user = query('SELECT id, name, email FROM users WHERE id=%s', (data['id'],), fetch=True, one=True)
```

```

        if not user:
            return jsonify({'error': 'Invalid user'}), 401

        return f(user, *args, **kwargs)
    except Exception:
        return jsonify({'error': 'Invalid token'}), 401

    return wrapper

```

8.2.4 Multinomial Naive Bayes Predictor (`backend/app.py`)

The algorithm used to categorize transaction descriptions:

```

import math
import re
from db import query

SMART_CATEGORIES = {
    'Food': ['swiggy', 'zomato', 'restaurant', 'cafe', 'food'],
    'Travel': ['uber', 'ola', 'metro', 'bus', 'train', 'flight'],
    'Shopping': ['amazon', 'flipkart', 'myntra', 'shopping'],
    'Medical': ['apollo', 'pharmacy', 'hospital', 'medicine'],
    'Entertainment': ['netflix', 'spotify', 'cinema', 'movie'],
    'Bills': ['electricity', 'broadband', 'recharge', 'bill', 'airtel', 'jio', 'vodafone', 'sms chgs'],
    'Groceries': ['grocery', 'groceries', 'supermarket', 'dmart', 'bigbasket'],
    'Education': ['college', 'school', 'course', 'udemy'],
    'Insurance': ['insurance', 'premium'],
    'Fuel': ['petrol', 'diesel', 'fuel'],
    'Salary': ['salary', 'payroll']
}

def ai_tokens(text):
    """Splits text into a list of alphanumeric tokens."""
    return re.findall(r'[a-z0-9]+', (text or '').lower())

def naive_bayes_category(uid, description):
    """Predicts a transaction category using Multinomial Naive Bayes classification."""
    tokens = ai_tokens(description)
    if not tokens:
        return {
            'category': 'Other',
            'confidence': 40,
            'matched': False,
            'algorithm': 'Multinomial Naive Bayes',
            'reason': 'No clear words found'
        }

    training = []
    # Load keywords
    for category, words in SMART_CATEGORIES.items():
        for word in words:
            training.append((category, word))

    # Load user transaction history
    history = query("""
        SELECT category, description, merchant FROM transactions
        WHERE user_id=%s AND type IN ('Income', 'Expense') AND category IS NOT NULL
        AND category<>'Transfer'
        ORDER BY transaction_date DESC, id DESC LIMIT 500
        """, (uid,), fetch=True)

    for row in history:
        text = ' '.join([str(row.get('description') or ''), str(row.get('merchant') or '')]).strip()
        if text:
            training.append((row['category'], text))

    if not training:
        return {

```

```

        'category': 'Other',
        'confidence': 40,
        'matched': False,
        'algorithm': 'Multinomial Naive Bayes',
        'reason': 'No training examples available'
    }

# Count frequencies
categories = sorted(set(category for category, _ in training))
doc_counts = {cat: 0 for cat in categories}
word_counts = {cat: {} for cat in categories}
total_words = {cat: 0 for cat in categories}
vocab = set()

for category, text in training:
    doc_counts[category] += 1
    for token in ai_tokens(text):
        vocab.add(token)
        word_counts[category][token] = word_counts[category].get(token, 0) + 1
        total_words[category] += 1

vocab_size = max(1, len(vocab))
total_docs = sum(doc_counts.values())
scores = {}

# Calculate probabilities
for category in categories:
    score = math.log((doc_counts[category] + 1) / (total_docs + len(categories)))
    denom = total_words[category] + vocab_size
    for token in tokens:
        score += math.log((word_counts[category].get(token, 0) + 1) / denom)
    scores[category] = score

# Get highest probability match
best = max(scores, key=scores.get)
max_score = scores[best]

probabilities = {cat: math.exp(sc - max_score) for cat, sc in scores.items()}
total_probability = sum(probabilities.values()) or 1
confidence = round(probabilities[best] / total_probability * 100)
matched_words = [token for token in tokens if token in vocab][:4]

if not matched_words or confidence < 45:
    return {
        'category': 'Other',
        'confidence': max(35, confidence),
        'matched': False,
        'algorithm': 'Multinomial Naive Bayes',
        'reason': 'No strong category pattern found'
    }

return {
    'category': best,
    'confidence': confidence,
    'matched': True,
    'algorithm': 'Multinomial Naive Bayes',
    'reason': 'Matched words: ' + ', '.join(matched_words)
}

```

8.3 Data Dictionary

This data dictionary outlines the relational schema configurations for tables managed in the MySQL database:

Table 1: `users`

Column	Data Type	Key	Null	Default	Description	Example
id	INT	PRIMARY	NO	AUTO_INCREMENT	Unique identifier for each user profile.	4
name	VARCHAR(100)	NO	NO		Full name of the user.	Rahul Sharma
email	VARCHAR(120)	NO	NO		Log-in email address.	student@example.com
password	VARCHAR(255)	NO	NO		Encrypted script password hash.	script:32768:8:1\$ab12...
created_at	TIMESTAMP	NO	NO	CURRENT_TIMESTAMP	Date and time the account was created.	2026-08-19 10:00:00

Tracks configured financial accounts and balances.

Table 3: `transactions`

Column	Data Type	Key	Null	Default	Description	Example
id	INT	PK	NO	AUTO_INCREMENT	Unique identifier for the transaction.	154
user_id	INT	FK	NO	-	Links the transaction to a user.	4
account_id	INT	FK	NO	-	Source account ID.	12
category	VARCHAR(100)	-	NO	-	Transaction class (Income/Expense/Transfer).	Expense
amount	DECIMAL(12,2)	-	NO	-	Category class (e.g., Food, Travel).	Food
description	VARCHAR(255)	-	YES	-	Numerical value of the transaction.	350.00
transaction_date	DATE	-	NO	-	Transaction details or memo text.	Swiggy Lunch Delivery
transfer_to_account	INT	-	YES	NULL	Date the transaction took place.	2026-08-19
review_status	ENUM	-	YES	-	Destination account ID for transfers.	14
	ENUM	-	YES	-	Status flag (Cleared/Pending/Flagged).	Cleared

Tracks budget limits against monthly expenses.

Table 5: `savings\_goals`

Column	Data Type	Key	Null	Default	Description	Example
id	INT	PK	NO	AUTO_INCREMENT	Unique identifier for the savings goal.	1
user_id	INT	FK	NO		Links the goal to a user.	4
goal_name	VARCHAR(100)		NO		User-defined name for the goal.	New Laptop
target_amount	DECIMAL(12,2)		NO		Financial savings target.	60000.00
saved_amount	DECIMAL(12,2)		YES	0.00	Current amount saved toward the goal.	15000.00
deadline	DATE		YES	NULL	Target completion date.	2026-12-31

FinWise AI

Books & Academic Sources

1. **Roger S. Pressman**, *Software Engineering: A Practitioner's Approach*, McGraw-Hill Education.
2. **Ian Sommerville**, *Software Engineering*, Pearson Education.
3. **Christopher D. Manning, Prabhakar Raghavan, & Hinrich Schütze**, *Introduction to Information Retrieval* (Chapter 13: Naive Bayes text classification), Cambridge University Press.
4. **Rajib Mall**, *Fundamentals of Software Engineering*, PHI Learning.
5. **E. Balagurusamy**, *Programming in Java*, Tata McGraw-Hill.

Web References

1. **React Official Documentation** - <https://react.dev> (Frontend framework reference).
2. **Bootstrap Web CSS Portal** - <https://getbootstrap.com> (Styling design reference).
3. **Flask micro-framework Documentation** - <https://flask.palletsprojects.com> (REST API routing reference).
4. **MySQL Database Documentation** - <https://dev.mysql.com/doc> (Database design reference).
5. **JSON Web Tokens Standard Specs** - <https://jwt.io> (Authorization token specifications).
6. **MDN Web Docs (JavaScript)** - <https://developer.mozilla.org> (Frontend programming reference).

FinWise AI Viva Notes

1. What is FinWise AI?

FinWise AI is a personal finance management system. It helps users track accounts, income, expenses, transfers, budgets, savings goals, statement imports, ledger entries, analytics, reports, and finance guidance.

2. Why is it called AI?

It has offline AI-style features:

- It predicts transaction category using Multinomial Naive Bayes.
- It answers finance questions using local rule-based intent matching.
- It gives finance insights from the user's own transaction data.

3. Does it need internet?

No. After setup, it can run locally without internet. The frontend, backend, database, finance chat, and category prediction all work on the local machine.

4. What algorithms are used?

- Multinomial Naive Bayes for transaction category prediction.
- Rule-based intent matching for offline finance chat.
- Ledger running balance algorithm.
- Duplicate transaction detection during statement import.
- Budget utilization calculation.
- Savings rate calculation.
- Goal progress calculation.
- Month-wise analytics grouping.
- Financial health rules.

5. Explain Multinomial Naive Bayes in this project.

The system reads a transaction description and predicts its category.

Example:

```
"salary july" -> Salary  
"swiggy dinner" -> Food  
"electricity bill" -> Bills
```

It calculates which category is most likely based on words in the description and previous transaction data.

6. Explain the ledger algorithm.

The ledger shows debit, credit, and running balance for each account.

$\text{Running Balance} = \text{Opening Balance} + \text{Credits} - \text{Debits}$

Income is a credit. Expense is a debit. Transfer creates a debit in one account and a credit in another account.

7. What is Balance Money?

Balance Money means the money currently available in accounts. It is the amount the user can actually use now.

8. What is Total Money?

Total Money means balance money plus money saved in goals.

$$\text{Total Money} = \text{Balance Money} + \text{Goal Savings}$$

9. Why are savings goals not shown as expenses?

Goal savings are not normal expenses because the user still owns the money. The money is only moved from an account into a savings goal. So it is tracked separately as savings.

10. How are transfers handled?

Transfers are used when money moves from one account to another account owned by the same user. A transfer is not income and not expense. It only changes account balances.

11. How should loans and credit cards be added?

Loans and credit cards can be added as account types. This keeps them visible with other accounts and makes tracking easier.

12. What is the use of bank statement import?

It saves time. The user can upload CSV, Excel, or text-based PDF statements, preview rows, fix categories, and import them into transactions.

13. How does duplicate import prevention work?

Before importing, the system checks if a similar transaction already exists for the same account, date, amount, type, and description. Duplicate rows are marked so the user does not import them again.

14. What is the use of Smart Tools?

Smart Tools are for repeated income or expense entries, like salary, rent, EMI, subscriptions, and bills. It helps avoid entering the same transaction manually every time.

15. Why was Family removed?

The project is kept simple and focused. The main purpose is personal finance tracking, so removing Family makes the system easier to understand and demonstrate.

16. What security is used?

- Password hashing.
- httpOnly cookie sessions.

-
- User-specific data filtering.
 - Hidden reset tokens by default.
 - Local environment variables for secrets.

17. What are the main modules?

- Login and Profile.
- Dashboard.
- Accounts.
- Transactions.
- Ledger.
- Budgets.
- Savings Goals.
- Analytics.
- Reports.
- Finance Chat.
- Smart Tools.

18. What is the database used?

MySQL is used to store users, accounts, transactions, budgets, goals, statement imports, automation entries, and preferences.

19. What is the limitation of the AI chat?

It is offline and explainable, but it is not a large online AI model. It answers from local project rules and finance data.

20. How do you run the project?

Open the project folder and run:

```
cd C:\FinWise_AI  
.\run.cmd
```

Then open:

```
http://127.0.0.1:5000
```

Keep the run window open.

FinWise AI Demo Checklist

Use this order when showing the project to your guide.

1. Start the App

```
cd C:\FinWise_AI
.\run.cmd
```

Open:

`http://127.0.0.1:5000`

2. Login

- Show login page.
- Login with your test account.
- Mention that passwords are stored as hashes.

3. Dashboard

Show these cards:

- Total Money.
- Balance Money.
- Income.
- Expense.
- Savings.

Explain:

`Total Money = Balance Money + Goal Savings`

4. Accounts

Show account types:

- Bank.
- Cash.
- UPI Wallet.
- Credit Card.
- Loan.
- Investment.

Add or show two bank accounts, one credit card, and one loan account.

5. Transactions

Show three transaction types:

-
- Income.
 - Expense.
 - Transfer.

Explain that transfer is used for moving money from one own account to another own account.

6. Ledger

Open Ledger and explain:

$\text{Running Balance} = \text{Opening Balance} + \text{Credits} - \text{Debits}$

Show how income, expense, and transfer affect debit and credit.

7. Bank Statement Import

Open Transactions and show Import Bank Statement.

Explain:

- CSV, Excel, and text-based PDF are supported.
- Preview appears before saving.
- Duplicates are marked before import.

8. Analytics

Open Analytics.

Show:

- Month selector.
- Income.
- Expense.
- Net flow.
- Saved to goals.
- Top category.
- Category chart.

Explain that the page filters transactions by selected month.

9. Budget

Open Budget.

Show category-wise budget limit and spending usage.

Explain:

$\text{Budget Usage Percent} = \text{Actual Expense} / \text{Budget Limit} * 100$

10. Savings Goals

Open Savings Goals.

Show:

-
- Goal target.
 - Saved amount.
 - Still needed.
 - Add savings to goal.

Explain that goal saving is not counted as normal expense because the money is still owned by the user.

11. Finance Chat

Open Finance Chat and ask these questions:

What algorithms are used here?
Is this offline?
What is balance money?
How does transfer work?
Explain ledger.

Mention that the answers come from local rules and local finance data.

12. Smart Tools

Open Smart Tools.

Show auto entries for repeated income or expenses such as:

- Salary.
- Rent.
- EMI.
- Bills.
- Subscription.

13. Profile

Open Profile.

Show:

- Name.
- Email.
- Password change.
- Theme selection.
- Logout button.

14. Reports

Open Reports.

Show export or report options available in the app.

15. Final Explanation

End with this short summary:

FinWise AI is an offline personal finance management system. It tracks accounts, income, expenses, transfers

Screenshot List For Report

Take screenshots of:

- Login page.
- Dashboard.
- Accounts page.
- Transactions page.
- Statement import preview.
- Ledger page.
- Analytics page.
- Budget page.
- Savings goals page.
- Finance chat page.
- Profile page.